

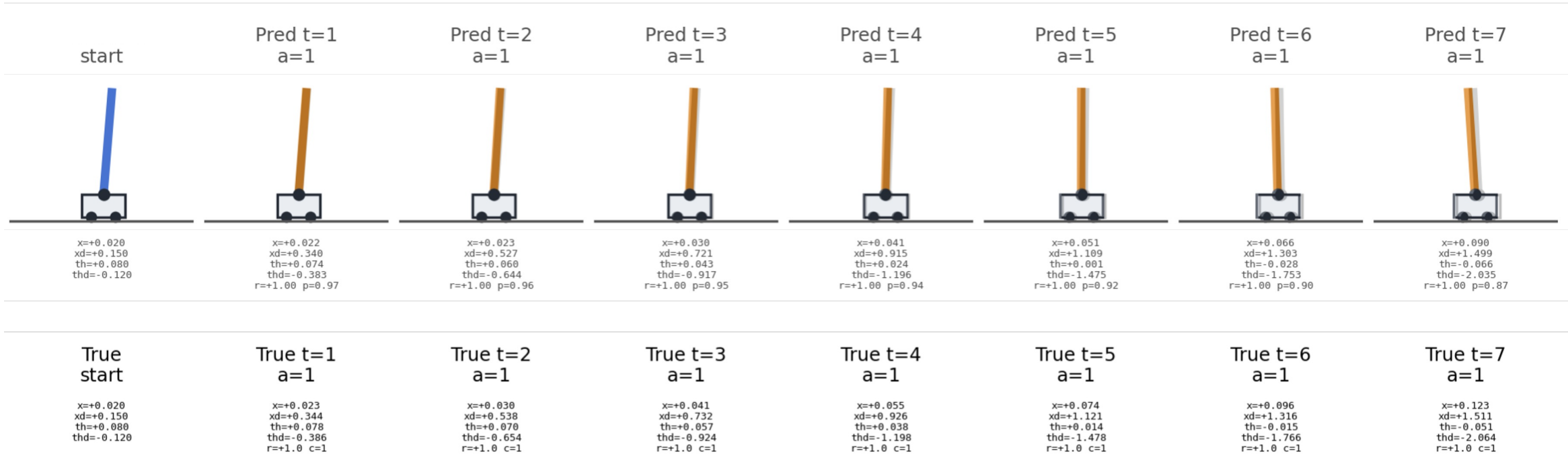


Sprint 2

13.05.26



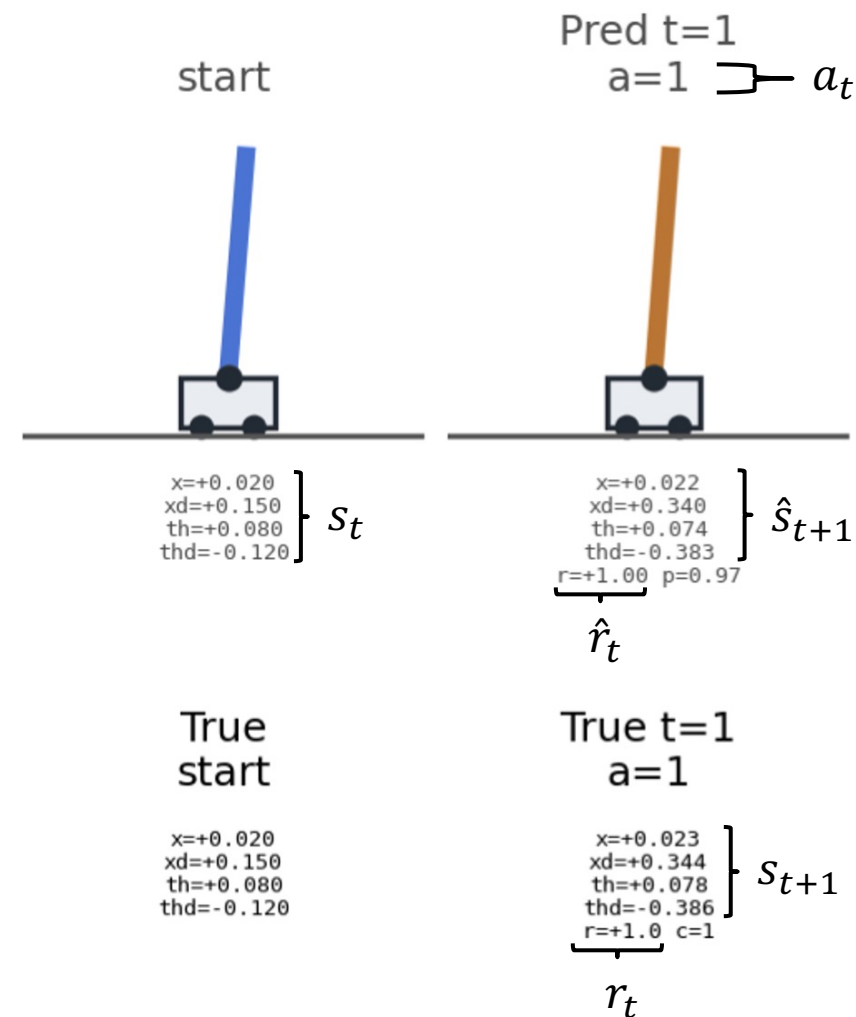
Learned dynamics model





Dynamics Model

$$f_{\theta}: s_t, a_t \rightarrow \hat{s}_{t+1}, \hat{r}_t$$





Ziele für Sprint 2

- Nutzt eure Algorithmen von Sprint 1, aber:
- Keine *wahre* Interaktion mit dem Environment (“env.step”)
- Sondern *gelernte* Interaktion mithilfe des Dynamics Modells
 - D.h., das Dynamics Modell muss vorher (gut!) gelernt werden
- Anwendung erst auf CartPole, dann FrozenLake



Pseudocode dynamics model

```
class DynamicsModel
    def __init__():
        layer1: 6x64 + ReLU
        layer2: 64x64 + ReLU
        s_head: linear 64x4
        r_head: linear 64x1
```



Pseudocode dynamics model

```
class DynamicsModel
    def __init__()
    def forward( $s_t$ ,  $a_t$ ):
        return  $\hat{s}_{t+1}$ ,  $\hat{r}_t$ 
```



Pseudocode dynamics model

```
class DynamicsModel
    def __init__()
    def forward( $s_t$ ,  $a_t$ )
    def loss( $s_t$ ,  $a_t$ ,  $s_{t+1}$ ,  $r_t$ ):
        generate  $\hat{s}_{t+1}$ ,  $\hat{r}_t$ 
        state loss =  $\text{MSE}(\hat{s}_{t+1}, s_{t+1})$ 
        reward loss =  $\text{MSE}(\hat{r}_t, r_t)$ 
        return state loss + reward loss
```



Training pseudocode

```
def collect_transitions(env, n_episodes):  
    for episode in range(n_episodes):  
        make random experiences  $s_t, a_t, s_{t+1}, r_t$   
    return buffer (all experience)
```




Training pseudocode

```
def train(n_training_steps):  
    initialize env, dynamics_model, optimizer  
    collect transitions  
    for step in range(n_training_steps):  
        take batch of data from buffer  
        calculate loss  
        update model parameters
```



Woher wissen wir obs funktioniert?

- Selber die vorhersagen angucken
- MSE ausgeben lassen